Robotic Technology HW#01

Amirhossein Ahmadinejad
401108686

***Abstract*—In this project, several ROS2[1] algorithms are developed to construct a two-wheel vehicle equipped with an IMU and a LIDAR sensor. First, we created the URDF[2] model to insert the vehicle into RVIZ and Gazebo. Then, we simulated a 2D LIDAR and an IMU[3] with Gaussian white noise. In the next stage, we implemented low-pass filtering, complementary filtering, bias correction, controllable motor commands, a motion-model-based controller, and a motion-model odometry node. Finally, we subscribed to smartphone IMU and magnetometer data and processed them using a low-pass filter, bias correction, a complementary filter, and accelerometer integration. The filtered motion data were validated by walking in a rectangular path and examining the results in RVIZ.**

I. Introduction

Mobile robots that rely on low-cost inertial and ranging sensors increasingly require robust state-estimation and control frameworks to achieve reliable navigation in real-world environments. ROS2 provides a flexible middleware for developing and integrating such systems, offering modularity, real-time communication capabilities, and a standardized toolchain for simulation and experimentation. However, accurately modeling sensor behavior, mitigating measurement noise, and fusing heterogeneous data sources remain key challenges, particularly for lightweight ground vehicles equipped with noisy IMUs and LiDAR sensors.

In response to these challenges, this work presents the development of a two-wheel differential-drive robot simulated in Gazebo and controlled through a set of custom ROS 2[1] nodes. The system includes a complete URDF model, simulated LiDAR and IMU sensors with Gaussian noise, signal-processing algorithms, motion-control modules, and odometry computation based on the vehicle's motion model. Beyond simulation, the system additionally integrates external smartphone IMU and magnetometer data, enabling evaluation of the implemented filtering and bias-correction pipelines on real sensor streams.

The objective of this project is to design and validate a modular sensor-processing and control architecture that helps us to get familiar with ROS2 and basic robotic codes. Through simulation experiments and real-world IMU tests, the proposed system demonstrates the effectiveness and Boundaries of complementary filtering, low-pass filtering, bias correction, and accelerometer integration in finding the rotation and position of the vehicle.

[1]Robot Operating System 2
[2]Unified Robotic Description Format
[3]Inertia Measure unit

II. Question One

A. Generating URDF File

For generating the robot, the latest URDF file from the GitHub repository was used, and several adjustments were applied, including modifying the base-link geometry, changing the wheel size, and positioning the wheel joints based on the base-link coordinates. All of these modifications were made according to the specification table provided in the assignment.

B. Using Correct Joints

After defining and placing the base link and wheels, the joint type required for vehicle motion is configured. Based on the group discussion, the joints are set to continuous, and no rotational constraints are applied.

C. Launching RVIZ

In this part, the launch file from the GitHub repository was used, and because it fully met the project requirements, no modifications were necessary. However, one adjustment in RViz is required: the fixed frame has to be set to $base\_link$ and the Robot Model should be added from $Add \longrightarrow By\ display\ type \longrightarrow RobotModel$ directory.

D. Saving the RVIZ file

In the last part of the question one, the RVIZ file is saved in config folder and is added to RVIZ node, so every time we run the command

```
ros2 launch robot_description display.launch.py
```

it will open this file.

III. Question two

A. Inserting sensors

After inserting the model, the LiDAR and IMU sensors were defined in the URDF. Their positions, orientations, and corresponding plugins are also configured. Through these steps, the sensors are fully specified within the simulation framework. An example of this configuration is shown below.

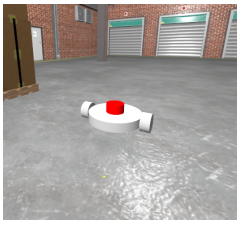

Fig. 1. Robot in Gazebo

- Setting size and rotation of IMU

```
<joint name="zed_imu_joint" type="fixed">
    <parent link="base_link"/>
    <child link="zed_imu_link"/>

    <origin xyz="0 0.0 0.12" rpy="0 0 0"/>
  </joint>

  <link name="zed_imu_link">
    <inertial>
      <mass value="0.05"/>`
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <inertia ixx="1e-5" ixy="0" ixz="0" iyy="1
          e-5" iyz="0" izz="1e-5"/>
    </inertial>
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <geometry>
        <box size="0.02 0.02 0.02"/>
      </geometry>
      <material name="imu_frame">
        <color rgba="1.0 1.0 0.0 1.0"/>
      </material>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <geometry>
        <box size="0.02 0.02 0.02"/>
      </geometry>
    </collision>
  </link>
```

- Adding IMU plugin

```
<!-- IMU Plugin -->
<gazebo>
  <plugin filename="libignition-gazebo-imu-
      system"
          name="ignition::gazebo::systems::Imu">
  </plugin>
</gazebo>

<gazebo reference="zed_imu_link">
  <sensor name="imu_sensor" type="imu">
    <always_on>1</always_on>
    <update_rate>1000</update_rate>
    <visualize>true</visualize>
    <topic>/zed/zed_node/imu/data_raw</topic>
```

B. Setting Up LIDAR, RVIZ, and Gazebo

Although the LIDAR sensor was fully defined in the URDF, several additional steps are required for proper operation in Gazebo and RVIZ. These steps are described below:

- A 2D LiDAR sensor is required for this project; however, the URDF initially contains a 3D LiDAR model. Therefore, the **vertical** scanning component is removed to convert it into a 2D LIDAR. The bridge that connected Gazebo to ROS2 and frame-ID converter nodes don't need any changes and they are correct on their own, remained unchanged, as they are already configured correctly.
- Next, the RVIZ workspace needed to be customized, similar to the procedure in Question 1. After configuring the environment, the LIDAR topic is added using
  $Add \longrightarrow By\ topic \longrightarrow PointCloud2$
  Then, the **Topic** parameter is set to `/points`, and the Reliability Policy is adjusted to **Best Effort**. Completing these steps allowed the LIDAR data to be visualized properly in RVIZ. Then we can save the RVIZ file and set the path in the launch file.

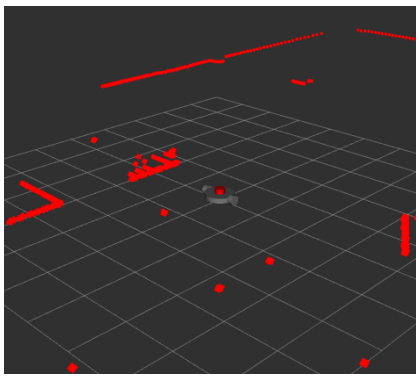

Fig. 2. LIDAR Map

IV. Question Three

At first a C++ package is built and then the following steps has been done.

A. Low Pass Filter

To reduce high-frequency noise in the inertial measurements, a dedicated ROS2 node named low pass filter is implemented. This node subscribes to raw IMU data, applies independent first-order low-pass filters to each accelerometer and gyroscope axis, and publishes the filtered measurements on a separate topic.

The node subscribes to the raw IMU topic `/zed/zed_node/imu/data_raw` and publishes the filtered IMU message on `/imu/filtered` .

For each of the six IMU channels (three linear accelerations and three angular velocities), the node implements a discrete-time first-order low-pass filter of the form

$$y[n] = \alpha\, x[n] + (1-\alpha)\, y[n-1], \tag{1}$$

where $x[n]$ is the raw measurement at sample $y[n]$ is the filtered output, and α is the filter coefficient. The coefficient α is computed directly from the desired cut-off frequency f_c and sampling frequency f_s as

$$\alpha = \frac{2\pi f_c}{2\pi f_c + f_s}. \tag{2}$$

Separate sampling and cut-off frequencies are defined for each channel:

- $f_{s,\mathrm{ax}}$, $f_{s,\mathrm{ay}}$, $f_{s,\mathrm{az}}$ and $f_{c,\mathrm{ax}}$, $f_{c,\mathrm{ay}}$, $f_{c,\mathrm{az}}$ for the three accelerometer axes,
- $f_{s,\mathrm{gx}}$, $f_{s,\mathrm{gy}}$, $f_{s,\mathrm{gz}}$ and $f_{c,\mathrm{gx}}$, $f_{c,\mathrm{gy}}$, $f_{c,\mathrm{gz}}$ for the three gyroscope axes.

These quantities are exposed as ROS2 parameters, which allows the filter characteristics to be tuned at run time without recompiling the node. In the default configuration, the accelerometer channels are assigned lower cut-off frequencies (e.g., $f_c = 10$ Hz), suitable for smoothing quasi-static motions,

while the gyroscope channels use higher cut-off frequencies (e.g., $f_c = 20$ Hz), preserving more of the rotational dynamics.

At the first received IMU sample, the filter states $y[0]$ for all channels are initialized to the raw values $x[0]$ to avoid a transient from zero:

$$y[0] = x[0] \tag{3}$$

For each subsequent message, the node:

1) extracts the raw linear accelerations (a_x, a_y, a_z) and angular velocities $(\omega_x, \omega_y, \omega_z)$,
2) applies the low-pass filter in (1) independently to each axis using the corresponding α from (2),
3) updates the stored previous outputs $y[n-1]$ with the newly computed $y[n]$.

A new IMU message is then constructed by copying the original message and replacing only the filtered fields:

- `linear_acceleration.x, .y, .z` are replaced by the filtered accelerations,
- `angular_velocity.x, .y, .z` are replaced by the filtered angular velocities.

The resulting message is published on the filtered IMU topic, providing a drop-in replacement for noisy raw IMU data in downstream nodes such as bias correction and complementary filter.

B. Bias Correction

In addition to low-pass filtering, the inertial measurements require bias removal to prevent drift in the continue modules. For this a ROS2 node named `bias_correct_imu` is implemented. This node estimates the static bias of both accelerometer and gyroscope channels and applies real-time correction to the incoming IMU data.

The node receives filtered IMU data from the topic `/imu/filtered` and publishes bias-corrected measurements on the topic `/imu/bias_corrected`.

Bias estimation is performed by averaging a fixed number of incoming IMU samples. The number of samples used for this procedure is defined by the parameter `bias_sample_count`, with a default value of 300. During this phase, for each incoming IMU message, the raw linear accelerations

$$(a_x, a_y, a_z)$$

and angular velocities

$$(\omega_x, \omega_y, \omega_z)$$

are accumulated. Once the total number of samples reaches the desired count, the accelerometer and gyroscope biases are computed as

$$b_{a_x} = \frac{1}{N}\sum_{i=1}^{N} a_x^{(i)}, \qquad b_{\omega_x} = \frac{1}{N}\sum_{i=1}^{N} \omega_x^{(i)}, \tag{4}$$

and likewise for the remaining axes.

The resulting biases

$$(b_{a_x}, b_{a_y}, b_{a_z}) \quad \text{and} \quad (b_{\omega_x}, b_{\omega_y}, b_{\omega_z})$$

represent the sensor offsets when the robot is stationary.

To prevent long-term drift and ensure that the bias estimate remains consistent with the sensor behavior, a periodic recalibration mechanism is integrated. Using a ROS2 wall timer, the bias estimation process is automatically re-triggered every `recalib_period_sec` seconds (default: 30 s). When triggered, the accumulated sums, sample counter, and bias flags are reset, and the node re-enters the bias-estimation phase.

Once the biases have been estimated, each incoming IMU message undergoes real-time correction. For a new linear acceleration measurement a_x, the corrected output is computed as

$$a_x^{\text{corr}} = a_x - b_{a_x}, \tag{5}$$

and equivalently for a_y, a_z, and the angular velocity components. A new IMU message is then created by copying the original message and replacing only the bias-corrected fields. All fields such as header, covariance matrices, and orientation remain unchanged to maintain compatibility with downstream nodes.

The corrected IMU data are finally published on the `/imu/bias_corrected` topic, providing a stable and drift-free input for subsequent processing stages.

C. Complementary Filter for Yaw Estimation

For Estimating yaw angle, a ROS2 node named `complementary_yaw_node` is implemented. This node fuses the gyro-based yaw rate with the orientation-derived yaw angle using a classical complementary filter. The method leverages the short-term accuracy of gyroscope integration and the long-term stability of the orientation measurement (typically provided by an onboard IMU fusion algorithm or magnetometer).

The node subscribes to bias-corrected IMU data on the topic `/imu/bias_corrected` and publishes the estimated orientation as a `QuaternionStamped` message on the topic `/estimation/orientation`. All topic names are configurable via ROS2 parameters.

Upon receiving the first IMU message, the node extracts the initial yaw angle from the IMU's orientation quaternion. The conversion from quaternion to roll, pitch, and yaw is performed using the standard transformation:

$$(\phi, \theta, \psi) = \mathrm{RPY}(q_x, q_y, q_z, q_w).$$

The initial yaw estimate ψ_0 is stored as the filter state. No integration or fusion is performed until the second message arrives, ensuring proper initialization.

For each subsequent IMU message, the timestamp is used to compute the sampling interval dt. The gyroscope's z-axis angular velocity ω_z (yaw rate) is integrated to obtain the predicted yaw angle:

$$\psi_{\text{gyro}} = \psi_{\text{est}}(n-1) + \omega_z\, dt. \tag{6}$$

Because yaw angles are periodic, the prediction is wrapped to the interval $[-\pi, \pi]$:

$$\psi_{\text{gyro}} = \mathrm{wrapToPi}(\psi_{\text{gyro}}).$$

Measurement Update: The IMU's orientation quaternion (which may include magnetometer- or filter-based heading estimation) is also converted into roll, pitch, and yaw. The measured yaw ψ_{meas} is likewise wrapped to $[-\pi, \pi]$.

The complementary filter computes the fused yaw estimate as

$$\psi_{\text{est}}(n) = \alpha\, \psi_{\text{gyro}} + (1 - \alpha)\, \psi_{\text{meas}}, \tag{7}$$

where $\alpha \in [0, 1]$ determines the contribution of the gyro prediction and the measurement. Typical values are $\alpha = 0.95$–0.99, giving strong weight to the short-term accuracy of gyroscope integration while allowing the measured yaw to correct long-term drift.

The resulting fused yaw angle is then wrapped to $[-\pi, \pi]$ and converted into a quaternion with roll and pitch set to zero:

$$q_{\text{out}} = \text{Quaternion}(0, 0, \psi_{\text{est}}).$$

D. Motor Command Distribution Interface

Before implementing the differential-drive controller, a dedicated motor command distribution node is developed to provide a clean interface for communicating wheel speeds to the robot's actuators. This node, `motor_command_node`, receives wheel RPM commands in a compact two-element array and publishes them onto two separate topics.

The node subscribes to the `/motor_commands` topic, which is expected to contain a `Float64MultiArray` message.

The node checks that the incoming array contains at least two elements. Insufficient data triggers a warning and prevents the publication of incomplete commands. When the message is valid, the left and right RPM values are assigned to individual `Float64` messages and published on:

- `/left_motor_rpm`
- `/right_motor_rpm`

E. Motor Controller

To convert high-level velocity commands into wheel actuation signals for the two-wheel differential-drive robot, a ROS2 node named `diff_drive_controller` is implemented. This node receives velocity commands in the standard `/cmd_vel` format and computes the corresponding wheel rotational speeds, which are then published as motor commands.

The node subscribes to `geometry_msgs/msg/Twist` messages on the topic `/cmd_vel`, where the relevant components are the linear velocity v in the x-direction and the angular velocity ω about the z-axis:

$$v = v_x = \texttt{msg.linear.x}, \qquad \omega = \omega_z = \texttt{msg.angular.z}.$$

Two geometric parameters are declared as ROS 2 parameters:

- the wheel radius r (`wheel_radius`),
- the wheelbase width b (`base_width`), i.e., the distance between the left and right wheel.

Under the standard differential-drive kinematic model, the linear and angular velocities of the robot are related to the right and left wheel linear velocities (v_r, v_l) by

$$v_r = v + \frac{\omega b}{2}, \qquad v_l = v - \frac{\omega b}{2}. \tag{8}$$

That these equations mentioned in the main and TA class.

These linear velocities are then converted to wheel angular velocities ω_r and ω_l using the wheel radius r:

$$\omega_r = \frac{v_r}{r}, \qquad \omega_l = \frac{v_l}{r}. \tag{9}$$

In the implementation, these angular velocities (in rad/s) are further converted to revolutions per minute (RPM) to match the expected motor command interface:

$$\text{RPM}_r = \omega_r \frac{60}{2\pi}, \qquad \text{RPM}_l = \omega_l \frac{60}{2\pi}. \tag{10}$$

The resulting motor commands are packaged into a `std_msgs/msg/Float64MultiArray` and published on the topic `/motor_commands`. The array contains two elements that is used for left and right wheel commands RPM.

F. Motion Model Based Odometry

To estimate the robot pose from wheel-based velocity measurements, a motion-model odometry node named `motion_model_odom` is implemented. This node integrates the linear and angular velocities obtained from a wheel encoder odometry source and publishes a continuous pose estimate in the `odom` frame, along with the corresponding TF transform between `odom` and `base_link`.

The node subscribes to `nav_msgs/msg/Odometry` messages on the topic `/wheel_encoder/odom`, which contains the robot twist information derived from the wheel encoders. The linear velocity v and angular velocity ω used by the motion model are extracted as

- v = `msg.twist.twist.linear.x`
- ω = `msg.twist.twist.angular.z`

The integrated odometry is then published on the topic `/motion_model/odom` and simultaneously broadcast as a TF transform.

Differential-Drive Motion Model: The robot pose is represented by (x, y, θ) in the `odom` frame, where (x, y) denotes the planar position and θ is the yaw angle. Under the standard differential-drive kinematics, the time evolution of the pose is governed by

$$\dot{x} = v \cos\theta, \qquad \dot{y} = v \sin\theta, \qquad \dot{\theta} = \omega. \tag{11}$$

These are motion model equations and these are extracted from the TA slides :

$$x_{k+1} = x_k + v_k \cos(\theta_k)\, \Delta t \tag{12}$$

$$y_{k+1} = y_k + v_k \sin(\theta_k)\, \Delta t \tag{13}$$

$$\theta_{k+1} = \theta_k + \omega_k\, \Delta t \tag{14}$$

where Δt is the elapsed time between two consecutive wheel-odometry messages. The node initializes (x, y, θ) to zero and updates them at each callback using the timestamp difference between the current and previous messages.

For each update, a `nav_msgs/msg/Odometry` message is populated with the current pose estimate:

- `odom.pose.pose.position.x = x`
- `odom.pose.pose.position.y = y`

- `odom.pose.pose.orientation` = θ

The header frame is set to `"odom"` and the child frame to `"base_link"`. The same information is also used to construct a `geometry_msgs/msg/TransformStamped` message, which is broadcast via a `tf2_ros::TransformBroadcaster`. This provides a consistent TF tree linking the fixed `odom` frame to the robot base frame `base_link`.

G. Implementing launch file

In the end many nodes are generated and running them one by one isn't a good option so a launch file is generated for simplifying the process and it's running command is;

```
ros2 launch robot_estimation estimator.launch.py
```

V. Question four

In Question 4, most components are identical to those in Question 3; therefore, additional explanations are omitted. Only the first part and RVIZ part are described in detail, and the corresponding launch-file command is provided at the end.

A. HTTP Based Smartphone IMU and Magnetometer in Real Time

In the final stage of the system, an additional ROS2 node is implemented to integrate inertial and magnetic field measurements from a smartphone into the ROS based processing pipeline. The node, named `http_imu_node`, acts as an HTTP server that receives JSON-formatted sensor data via HTTP `POST` requests and publishes them as standard ROS2 IMU and magnetometer messages.

The node creates two publishers:

- `phone/imu` (`sensor_msgs/msg/Imu`)
- `phone/magnetometer` (`sensor_msgs/msg/MagneticField`)

The corresponding message headers use a configurable frame ID (default: `phone_imu`), and timestamps are generated from the node's clock at reception time.

An embedded HTTP server (based on `httplib`) is started on `0.0.0.0` and listens on a configurable port (default: 8000). The main data endpoint is exposed at the route `/data`, which accepts HTTP `POST` requests containing a JSON body.

Incoming HTTP requests are parsed using the `nlohmann::json` library. Since the smartphone payload may contain nested objects and arrays, a recursive visitor function is employed to traverse the entire JSON structure. For each object that contains both `name` and `values` fields, the node checks the sensor type and extracts the corresponding components:

- **Gyroscope**
- **Accelerometer**
- **Magnetometer**

For the magnetometer, the phone is assumed to report measurements in microtesla (μT), consistent with common smartphone APIs. These are converted to tesla (T) for the `sensor_msgs/msg/MagneticField` message using:

$$B_{[\mathrm{T}]} = B_{[\mu\mathrm{T}]} \times 10^{-6}.$$

Boolean flags (`has_gyro`, `has_accel`, `has_mag`) track which sensor types are present in the current JSON payload. If at least one of gyroscope or accelerometer data is found, an IMU message is published; if magnetometer data is found, a magnetic field message is published. In the case where the JSON is valid but no recognized sensors are found, a warning is logged.

To prevent blocking the ROS2 executor, the HTTP server's `listen()` loop is executed in a separate thread. The node destructor gracefully stops the server and joins the thread, ensuring clean shutdown.

B. RVIZ and Launch File

In the end for displaying the position in RVIZ and running all the nodes at the same time a launch file was generated and we can launch it by using this command:

```
ros2 launch robot_nav nav.launch.py
```

But because of the unknown reasons when we launch the file it doesn't show the path, so if you want to see the path it's better to run the nodes one by one and after that run:

```
rviz2
```

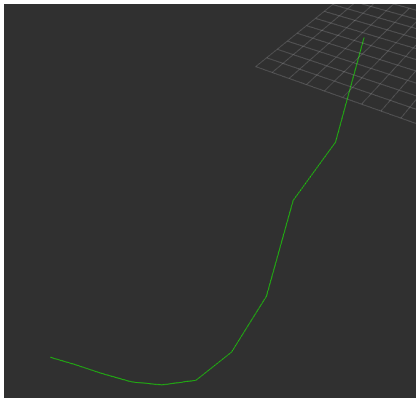

Fig. 3. Output of sensors in RVIZ